

tic tac toe

$nr \leftarrow 3, nc \leftarrow 4, size \leftarrow nr \cdot nc.$

1. state

1.2. $M \leftarrow M(nr, nc) \leftarrow 0$

M is data, state, $M_{ij} \in \{-1, 0, 1\}$

$f(M) = h$, f is bijective. h is hash value.

1.2.1 f : $x \leftarrow 0$

$\forall i \in M, x \leftarrow 3x + (i+1)$

$\therefore h \leftarrow x$

1.2.2 $\therefore h$ is ternary number:

$(M_{1,1}, M_{1,2}, \dots, M_{nr,nc+1})$

1.3 is end

if $end \neq -1$, return end.

// $end \in \{-1, 0, 1\}$

sequence $re \leftarrow \emptyset$.

$\forall i \in [1, nr]$,

re add $sum(M_{i,:})$

$\forall i \in [1, nc]$,

re add $sum(M_{:,i})$

if $nr = nc$

$t \leftarrow trace(M)$

$rt \leftarrow reverse\ trace(M)$

re add t .

re add rt .

$\forall i \in [1, |re|]$

$x \leftarrow re(i)$

$x_0 \leftarrow \begin{cases} nc & \text{if } i \leq nr \\ nr, & \text{else} \end{cases}$

if $x = x_0$

$w \leftarrow 1, end \leftarrow 1, stop$

if $x = -x_0$

$w \leftarrow -1, end \leftarrow 1, stop$

if $\|M\|_1 = size$

$w \leftarrow 0, end \leftarrow 1, stop$

$end \leftarrow 0$.

1.4. next state $(i, j, sb: symbol)$

$s' \leftarrow state()$

$s': M \leftarrow M$

$s': M_{ij} \leftarrow sb$.

1.5 print state

init

$M \leftarrow 0, \in \{-1, 0, 1\}^{nr \times nc}$

$h \leftarrow -1, \in \{-1\} \cup \{0, 1, \dots, 3^{size} - 1\}$

$w: winner \leftarrow 0 \in \{-1, 0, 1\}$

$end \leftarrow -1 \in \{-1, 0, 1\}$

2. all states S .

2.3 get all states:

csb: current symbol $\leftarrow 1$.

// 1 moves first, -1 second.

$s \leftarrow \text{state}()$

S : all states $\leftarrow \text{dict}$

S add (s .hash, s)

get all states impl (s , cs, S)

2.4 get all states impl (s , cs, S)

$\forall i \in [1, nr]$,

$\forall j \in [1, nc]$,

if $s.M_{ij} = 0$

then $s' \leftarrow s$.next state (i, j , cs)

$h' \leftarrow s'$.hash

if $h' \notin S$

end' $\leftarrow s'$ is end

S add (h' , s')

if end' = 0

get all states impl (s' , -cs, S) if is print state

3. Judge (Player $p1$, $p2$).

3.2 init:

$p1 \leftarrow p1$, $p2 \leftarrow p2$.

cp: current player $\leftarrow 0 \in \{-1, 0, 1\}$

~~psb1~~: $p1$ symbol $\leftarrow 1$

~~psb2~~: $\leftarrow -1$

$p1$ set symbol (~~psb1~~)

$p2$ set symbol (~~psb2~~)

3.3. reset:

$p1$ reset

$p2$ reset

3.4 alternate:

loop:

yield $p1$

yield $p2$.

3.5 play (is print state $\leftarrow 0$)

alt $\leftarrow$ alternate

reset

$s \leftarrow \text{State}$

$p1$ set state s

$p2$ set state s

if is print state

s print

loop:

player $p \leftarrow \text{next}(\text{alt})$

page 2.

$i, j, sb \leftarrow p \text{ act}$

$s' \leftarrow s, \text{next state}(i, j, sb)$

$h' \leftarrow s' \text{ hash}$

$s \leftarrow B(h')$ // get s from S .

// not use s' , ' i ' s has some

// values computed already.

p_1 set state s .

p_2 set state s .

if i is print state

s print

if s is end

return s , winner.

~~end loop.~~

4. AI player

4.2 init (α : step size $\leftarrow 0.1$,

$\epsilon \leftarrow 0.1$)

V : estimations $\leftarrow$ dict

$\alpha \leftarrow \alpha, \epsilon \leftarrow \epsilon$

Sequence $S_t \leftarrow \emptyset$

Seq greedy $g \leftarrow \emptyset$

symbol $sb \leftarrow 0$

4.4 reset

$S_t \leftarrow \emptyset$

$g \leftarrow \emptyset$

4.5 set state (s)

S_t add s ,

g add 1.

4.6 set symbol (sb)

// compute $V(s)$ initially.

$sb \leftarrow sb$.

$\forall h \in S$

$s \leftarrow S(h)$

if s .end

if s .winner = sb

$V(h) \leftarrow 1$

else if s .winner = 0

$V(h) \leftarrow 0.5$

else

$V(h) \leftarrow 0$

else

$V(h) \leftarrow 0.5$

page 3.

4.7 backup:

states hash $hs \leftarrow \{s.hash \mid s \in S_t\}$

// $h \Leftrightarrow s$

$\forall i \in \{1, \dots, |hs|-1, |hs|\}$

$s \leftarrow hs(i), s' \leftarrow hs(i+1)$

$td_{error} \leftarrow g(i)(V(s') - V(s))$

$V(s) \leftarrow V(s) + \alpha \delta$

$V_t = \{V(s) \mid s \in S_t\}$

$i \leftarrow \text{argmax}_i V_t(i)$ randomly

// if $\exists$ multiple i , choose

// randomly.

$a \leftarrow as(i)$

return (a, sb)

4.8 act

// choose a based on s .

$s \leftarrow S_t(-1)$

$as : \text{next action set} \leftarrow \emptyset$

$ss : \text{next state set} \leftarrow \emptyset$

$\forall i \in [1, nr]$

$\forall j \in [1, nc]$

if $s.M(i, j) = 0$

as add (i, j)

$s' \leftarrow s, \text{next state}(i, j, sb)$

ss add (s', hash)

$ran \leftarrow \text{rand}()$

if $ran < \epsilon$

choose i from $[1, |as|]$ randomly.

$a \leftarrow as(i)$, ~~and~~ $g(-1) \leftarrow 0$

return (a, sb)

else

4.9 save policy.

4.10 load policy

5. human player

5.2 init

$sb \leftarrow 0$

$keys \leftarrow \{'q', 'w', 'e', \dots\}$

state $s \leftarrow \text{state}$

5.3 reset

pass

5.4 set state (s)

$s \leftarrow s$

5.5 set symbol (sb)

$sb \leftarrow sb$

~~5.6 act :~~

~~5.7 print~~

page 4.

5. to act

```
key ← input()
n ← keys.index(key)
i ← (n-1)/nc + 1 (n-1)/nc + 1
j ← (n-1)%nc + 1
return (i, j, sb).
```

6. train (epochs, print every n ← 500) $\forall i \in [1, \text{turns}]$

```
p1 ← A1 player ( $\epsilon \leftarrow 0.01$ )
p2 ← A1 player ( $\epsilon \leftarrow 0.01$ )
j ← judger (p1, p2)
p1 win ← 0.0
p2 win ← 0.0
```

$\forall i \in [1, \text{epochs}]$

winner w ← j play (is print ← 0)

if w = 1
p1 win ++

if w = -1
p2 win ++

if i % print every n = 0

print (i, p1 win / i, p2 win / i)

p1 backup, p2 backup, j reset.

end $\forall$
p1 ~~backup~~ save policy, p2 ~~backup~~ save policy.

7. compete (turns)

p1 ← A1 player ($\epsilon \leftarrow 0$)

p2 ← A1 player ($\epsilon \leftarrow 0$)

j ← judger (p1, p2)

p1 load policy, p2 load policy.

p1 win ← 0, p2 win ← 0

w ← j play

if w = 1, p1 win ++

if w = -1, p2 win ++

j reset.

end $\forall$.

print (turns, p1 win / turns, p2 win / turns)

8 play (human ~~choice~~ ^{symbol} ← 1)
loop: hsb: $\in \{1, -1\}$

if hsb = 1, p1 ← human player

p2 ← A1 player

else p1 ← A1 player, p2 ← human player.

p1 load policy, p2 load policy.

j ← judger (p1, p2)

w ← j play.

if w = hsb print (you win)

else if w = -hsb print (you lose)

else print (tie).

9. all process:
S ← get all states
train (10^5)
compete (10^3)
play

1. Bandit

1.3 init($k \leftarrow 10, \epsilon \leftarrow 0, \text{initial} = 0,$
 $\alpha = 0.1, \text{is_using_sample_average} \leftarrow 0,$
 $c \leftarrow 0, \text{is_using_gradient} \leftarrow 0,$
 $\text{is_using_gradient_baseline} \leftarrow 0,$
 $\text{true_reward} \leftarrow 1$)
 $A_s \leftarrow A_s \text{ letters} \in \mathcal{N}^k$
 $\text{time } t \leftarrow 0$
 $\bar{R} \leftarrow 0.$

1.4 reset

$\forall a \in \mathcal{A} \ q^* \sim \mathcal{N}(tr, 1) \in \mathbb{R}^k$
 $Q \leftarrow \text{initial}, \in \mathbb{R}^k$
 $N \leftarrow 0, \in \mathbb{N}^k$
 $A^* \leftarrow \underset{a}{\operatorname{argmax}} q^*(a)$
 $t \leftarrow 0$

1.5 act

① if $\text{rand}() < \epsilon$
 $\text{return } A \leftarrow A_s \text{ randomly.}$

// (A_s is vector, but here is seen as a set)

② if $c \neq 0, (\text{i.e., } c > 0)$

if $t < k, \text{ if } N(a) = 0. \text{return } A \leftarrow a, \forall a.$

else $Q_{\text{UCB}}(a) \leftarrow Q(a) + c \sqrt{\frac{\log t}{N(a)}}, \forall a$

$\text{return } A \leftarrow \underset{a}{\operatorname{argmax}} Q_{\text{UCB}}(a) \text{ randomly.}$

③ if is using gradient

$H \leftarrow Q$

$e^H \leftarrow e^H$

$\pi \leftarrow \frac{e^H}{\sum e^H} \in \mathbb{R}^k$

$\text{return } A \leftarrow \text{draw from } A_s$
 $\text{by } \pi.$

④ $A \leftarrow \underset{a}{\operatorname{argmax}} Q(a) \text{ randomly.}$
 $\text{return } A^a.$

1.6 step (action: A)

$R \leftarrow \mathcal{N}(q^*(A), 1),$

$t \leftarrow t+1, N(A) \leftarrow N(A)+1.$

$\bar{R} \leftarrow \bar{R} + \frac{1}{t}(R - \bar{R})$

if is sample average

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}(R - Q(A))$

else if is using gradient

$\text{onehot} \leftarrow 0, \in \mathbb{N}^k$

$\text{onehot}(A) \leftarrow 1.$

if is u.g.b., $\text{baseline} \leftarrow \bar{R}$

else $\text{baseline} \leftarrow 0.$

$H \leftarrow H + \alpha(R - \text{baseline})(\text{onehot} - \pi)$

$Q \leftarrow H$
 $Q(A) \leftarrow Q(A) + \alpha(R - Q(A))$

$\text{return } R.$

2. simulate (runs $\leftarrow$ 2000, time $\leftarrow$ 1000, bandits)

rewards $rs \leftarrow 0 \in \mathbb{R}^{|bandits| \times runs \times time}$

best action counts $bac \leftarrow 0$, same shape with rs .

~~for~~ $\forall i, bandit \in bandits$

$\forall run \in [1, runs],$

bandit.reset

$\forall t \in [1, time]$

$a \leftarrow bandit.act$

$r \leftarrow bandit.step(a)$

$rs(i, run, t) \leftarrow r$

if $a = bandit.best_action.A^*$

$bac(i, run, t) \leftarrow 1$

end $\forall t$.

end $\forall i, run$.

mean best action counts $mbac \leftarrow bac.mean(axis \leftarrow 2)$

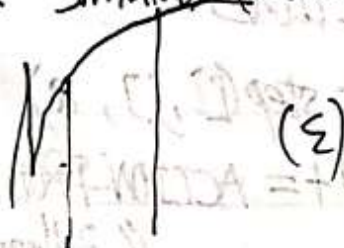
mean rewards $mrs \leftarrow rs.mean(axis \leftarrow 2)$

return $mbac, mrs, \in \mathbb{R}^{|bandits| \times time}$.

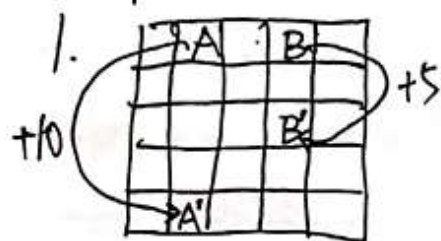
3. bandit $\leftarrow$ Bandit ($\epsilon=0$, UCB para $c \leftarrow 2$,
is_using_sample_average $\leftarrow 1$)

average rewards $rs \leftarrow simulate(2000, 1000, bandit)$

plot(rs): UCB:



Gridworld



$$2. X_m \leftarrow 5, Y_m \leftarrow 5$$

$$\text{Pos}_A \leftarrow (2, 5)$$

$$\text{Pos}_{A'} \leftarrow (2, 1)$$

$$\text{Pos}_B \leftarrow (4, 5)$$

$$\text{Pos}_{B'} = (4, 3)$$

$$R_A = 10, R_B = 5$$

$$\gamma = 0.9$$

$$A = \{(-1, 0), (0, 1), (1, 0), (0, -1)\}$$

$$\text{Action_Pr} \leftarrow 0.25$$

2.3 step (states, action a)

if $s = \text{Pos}_A$

return $\text{Pos}_{A'}, R_A$.

if $s = \text{Pos}_B$

return $\text{Pos}_{B'}, R_B$

$$s' \leftarrow s + a$$

$$x, y \leftarrow s'$$

if $x < 1$ or $x > X_m$ or $y < 1$ or $y > Y_m$

$$\text{reward } r \leftarrow -1$$

$$s' \leftarrow s$$

else $r \leftarrow 0$
return s', r .

$$1.3 \text{ reward} \in \{-1, 0, 5, 10\}$$

action $\in \{\text{left, up, right, down}\}$

1.4 suppose the agent selects all four actions with equal probability in all states.

v_π is the value function for this random policy, for the discounted reward case with $\gamma = 0.9$.

v_π can be computed by solving the system of linear equations,

i.e., Bellman equation for v_π .

$$v_\pi(s) \doteq \mathbb{E}_\pi(G_t | S_t = s)$$

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) (r + \gamma v_\pi(s')), \quad \forall s \in S$$

1.5 let v_* be optimal value function.

We can solve the Bellman optimality equation for v_* .

$$v_*(s) = \max_{a \in A(s)} \sum_{s', r} p(s', r | s, a) (r + \gamma v_*(s'))$$

3. figure 3.2()

Mat $v \leftarrow 0, \in \mathbb{R}^{X_m \times Y_m}$

Mat $v' \leftarrow v$

loop:

$v' \leftarrow 0$

$\forall (i, j) \in [1, X_m] \times [1, Y_m],$

$s \leftarrow (i, j)$

$\forall a \in A$

$s', r \leftarrow \text{step}(s, a)$

$\pi_{a|s} \leftarrow \text{Action-Pr}$

$v'(s) += \pi_{a|s} \cdot (r + \gamma v(s'))$

// Bellman equation for v_t

end $\forall a$

end $\forall i, j$.

if $\|v - v'\|_1 < 10^{-4}$

draw v .

stop

$v \leftarrow v'$

4. figure 3.5()

Mat $v \leftarrow 0(X_m, Y_m)$

Mat $v' \leftarrow v$

$vs \leftarrow 0 \in \mathbb{R}^{|A|}$

loop:

$v' \leftarrow 0$

$\forall s = (i, j) \in [1, X_m] \times [1, Y_m]$

$vs \leftarrow 0$

$\forall a \in A$

$s', r \leftarrow \text{step}(s, a)$

$vs(a) \leftarrow r + \gamma v(s')$

end $\forall a$

$v'(s) \leftarrow \max_a (vs(a))$

// value iteration.

end $\forall s$.

if $\|v - v'\|_1 < 10^{-4}$

draw v

stop

$v \leftarrow v'$

5. draw v :

$v_draw \leftarrow v$.

v_draw rotate 90° clockwise

v_draw ^{axis} show.

Gambler's problem.

1.

The problem can be formulated as an undiscounted, episodic, finite MDP. The argmax action selection with respect to the optimal value function.

$$s \in S = \{1, 2, \dots, 99\} \cup \{0, 100\}$$

$s=0$ and $s=100$ are two dummy states corresponding to termination, giving them

values of 0 and 1 respectively.

$$a \in A(s) = \{0, 1, \dots, \min(s, 100-s)\}$$

$$r \in \{0, 1\}.$$

$$S^+ = \{1, 2, \dots, 99\},$$

$$S^+ \cup \{0, 100\} = S.$$

$$2. \text{Goal} = 100,$$

$$S = \{0, 1, \dots, \text{Goal}\},$$

$$S^+ = \{1, 2, \dots, \text{Goal}-1\}$$

$$p_h = 0.4$$

3. figure 4.3()

1.3 the state-value function $V(s), s \in S$

gives the probability of winning from each state.

Map $V: S \rightarrow \mathbb{R},$

$$V(S) \leftarrow 0, V(\text{Goal}) = 1$$

$$\text{sweeps_history} \leftarrow \emptyset$$

The optimal policy maximizes the probability of reaching the goal.

loop: // value iteration

$$\text{sweeps_history.add}(V)$$

$$\Delta \leftarrow 0$$

1.4.

let p_h denote the probability of the coin coming up heads. If p_h is known, the

$$\forall s \in S^+$$

$$v \leftarrow V(s)$$

the entire problem is known and it can be solved, for instance, by value iteration.

$$\text{Vector } A \leftarrow A(s) = \{0, \dots,$$

$$\min(s, \text{Goal}-s)\}$$

1.5 for the case of $p_h = 0.4$, there is

$$\text{Vector } v \in \mathbb{R}^{|A|}$$

a whole family of optimal policies,

$$\forall i_a \in |A|, (\text{here we mean } i_a \in \{1, \dots, |A|\})$$

$a \leftarrow A(i_a)$	figure:
$us(i_a) \leftarrow p_h \cdot V(s+a) + (1-p_h)V(s-a)$	$\forall V \in \text{sweep_history}$
end $\forall i_a$	plot V .
$V(s) \leftarrow \max_{i_a} (us)$	xlabel (Capital)
$\Delta \leftarrow \max(\Delta, v - V(s))$	ylabel (value estimations)
end $\forall s$.	plot(π)
if $\Delta < 10^{-9}$: sweep_history add V . break loop.	xlabel (Capital)
end loop.	ylabel (final policy)

// compute optimal policy.

Map $\pi : S \rightarrow N$,

$\pi(s) \leftarrow 0$.

$\forall s \in S^+$

Vec $A \leftarrow A(s) = \{0, 1, \dots, \min(s, \text{Goal} - s)\}$

Vec $us \leftarrow 0, \in \mathbb{R}^{|A|}$

$\forall i_a \in |A|$

$a \leftarrow A(i_a)$

$us(i_a) \leftarrow p_h \cdot V(s+a) + (1-p_h) \cdot V(s-a)$

end $\forall i_a$
 $\pi(s) \leftarrow A(\arg\max_{i_a} (us(i_a))) + 1$

// avoid choosing $a=0$ which doesn't change state.
 end $\forall s$.

Chapter 4

1. Value iteration

$$V = 0 \in \mathbb{R}^{|S|}$$

loop:

$$\Delta \leftarrow 0$$

$$\forall s \in S$$

$$v \leftarrow V(s).$$

$$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) (r + \gamma V(s')) \text{ end } \forall s$$

$$\Delta \leftarrow \max(\Delta, |v - V(s)|)$$

end $\forall s$.

if $\Delta < \theta$ (a small positive number)

stop loop

end loop

$$\therefore \pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) (r + \gamma V(s'))$$

2. correction: Gambler's problem

$$S = \{1, 2, \dots, 99\}$$

$$S^+ = S \cup \{0, 100\}$$

$$\text{i.e. } S \leftrightarrow S^+$$

3. Policy iteration (using iterative policy evaluation)

1) $V(s) \in \mathbb{R}, \pi(s) \in A(s), \forall s \in S$. go to 2).

initialize V, π arbitrarily.

2) policy evaluation

loop:

$$\Delta \leftarrow 0$$

$$\forall s \in S$$

$$v \leftarrow V(s)$$

$$V(s) \leftarrow \sum_{s', r} p(s', r | s, \pi(s)) (r + \gamma V(s'))$$

$$\Delta \leftarrow \max(\Delta, |v - V(s)|)$$

if $\Delta < \theta$, stop loop

3) policy improvement

policy stable $\leftarrow 1$

$$\forall s \in S$$

$$\text{old } a \leftarrow \pi(s)$$

$$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) (r + \gamma V(s'))$$

if old $a \neq \pi(s) : \text{policy stable} \leftarrow 0$

end $\forall s$.

if policy stable

stop and return $V \approx v_*, \pi \approx \pi_*$.

Car rental

1. max cars at 1: $M_1 = 20$

max cars at 2: $M_2 = 20$.

max move: $M_m = 5$

expected rental request at i : λ_{i1}

expected ~~rental~~ returns at i : λ_{i2}
 $i = 1, 2$.

$$\therefore \begin{pmatrix} \lambda_{11} & \lambda_{12} \\ \lambda_{21} & \lambda_{22} \end{pmatrix} = \begin{pmatrix} 3 & 3 \\ 4 & 2 \end{pmatrix}$$

discount $\gamma = 0.9$

rent reward $R_r = R_{rent} = 10$,

move cost $c_m = 2$.

$A = \{-M_m, \dots, M_m\}$

Poisson upper bound $M_p = 11$

poisson cache: dict

2 poisson $pr(n, \lambda)$ // $\lambda < 10$

key = $n \cdot 10 + \lambda + 1$

if key $\notin$ poisson cache pc

pc(key) $\leftarrow$ poisson.pmf(n, λ) $\therefore n_{1v} \leftarrow \min(n'_1, req_1)$

return pc(key)

2.3 poisson.pmf:

$$f(n, \lambda) = e^{-\lambda} \cdot \lambda^n / n!$$

$$// e^\lambda = \sum_{n=0}^{\infty} \frac{\lambda^n}{n!}$$

3. Bellman equation for q_i

Bellman for q -s.a:

input: s, a, V , is constant returned cars

output:

$$q(s, a) \leftarrow E_a \leftarrow \sum_{s', r} p(s', r | s, a) (r + \gamma V(s'))$$

process:

expected return: $E_a \leftarrow 0$

$$E_a \leftarrow E_a - |a| \cdot c_m$$

number of cars at i : n_i

$$n_1 \leftarrow \min(s_1 - a_1, M_1)$$

$$n_2 \leftarrow \min(s_2 + a_2, M_2)$$

$\forall$ rental request $req_1 \in \{0, \dots, M_p\}$

$\forall req_2 \in \{0, \dots, M_p\}$

$$p \leftarrow \text{poisson_pr}(req_1, \lambda_{11})$$

$$\cdot \text{poisson_pr}(req_2, \lambda_{21})$$

valid rental at i : n_{iv}

$$n'_1 \leftarrow n_1, n'_2 \leftarrow n_2$$

$$n_{2v} \leftarrow \min(n'_2, req_2)$$

$$r \leftarrow (n_{1v} + n_{2v}) R_r$$

$$n'_1 \leftarrow n'_1 - n_{1v}, n'_2 \leftarrow n'_2 - n_{2v}$$

if is constant returned cars
 returned cars $ret_1 \leftarrow \lambda_1$
 $ret_2 \leftarrow \lambda_2$
 $n'_1 \leftarrow \min(n_1 + ret_1, M_1)$
 $n'_2 \leftarrow \min(n_2 + ret_2, M_2)$
 $s' \leftarrow (n'_1, n'_2)$
 $E_a \leftarrow E_a + p(r + \gamma V(s'))$

else
 // $E_a \leftarrow E_a + p(r + \gamma \sum_s p' V(s'))$

$E'_a \leftarrow 0$

$\forall ret_1, ret_2 \in \{0, \dots, M_p\}$

$p' \leftarrow \text{poisson_pr}(ret_1, \lambda_1) \cdot \text{poisson_pr}(ret_2, \lambda_2)$

$n'_1 \leftarrow \min(n_1 + ret_1, M_1)$

$n'_2 \leftarrow \min(n_2 + ret_2, M_2)$

$s' \leftarrow (n'_1, n'_2)$

$E'_a \leftarrow E'_a + p' V(s')$

end $\forall ret_1, ret_2$.

$\therefore E_a \leftarrow E_a + p(r + \gamma E'_a)$

end $\forall req_1, req_2$.

return E_a .

Map $V: S \rightarrow R$, $V(s) \leftarrow 0$

Map $\pi: S \rightarrow A \subseteq Z$, $\pi(s) \leftarrow 0$

iteration $k \leftarrow 0$

loop:

draw policy π

loop: (policy evaluation (in-place))

$\Delta \leftarrow 0$

$\forall s \in S$, $v \leftarrow V(s)$,

$V(s) \leftarrow \text{Bellman_for_q_sa}$

$(s, \pi(s), V, \text{is constant})$

// $V(s) \leftarrow q(s, \pi(s))$

$\Delta \leftarrow \Delta \vee |v - V(s)|$

end $\forall s$.

if $\Delta < 10^{-4}$: stop loop.

end loop. // inner loop.

is policy stable $\leftarrow 1$.

$\forall s \in S$, $a_{old} \leftarrow \pi(s)$

$qsa \in R^{|A|}$

$\forall i_a \in \{1, \dots, |A|\}$

$a \leftarrow A_{i_a}$

4. figure 4-2 (is constant returned cars ≤ 1)

if $a \in [-s_2, 0] \cup [0, s_1]$

$q_{sa}(i_a) \leftarrow \text{Bellman for } q_{sa}(s, a, V, \text{is constant})$

else

$q_{sa}(i_a) \leftarrow -\infty$

end $\forall i_a$

$\pi(s) \leftarrow A(\text{argmax}(q_{sa}))$

if is policy stable and $a_{old} \neq \pi(s)$

is policy stable $\leftarrow 0$

end $\forall s$

if is policy stable

draw V , stop

$k \leftarrow k+1$

5. We formulate this as a
continuing finite MDP.

3.

Car rental (synchronous)

1. $M_1=20, M_2=20, c_m=2$.

additional park cost $c_{park}=4$

$R_r=10$, rent reward.

$$\begin{pmatrix} \lambda_{11} & \lambda_{12} \\ \lambda_{21} & \lambda_{22} \end{pmatrix} = \begin{pmatrix} 3 & 3 \\ 4 & 2 \end{pmatrix}$$

$$S \leftarrow \{0, \dots, M_1\} \times \{0, \dots, M_2\}$$

poisson cache pc : dict

2. poisson(n, λ)

$$key \leftarrow n \cdot 10 + \lambda$$

if $key \notin pc$

$$pc(key) \leftarrow e^{-\lambda} \cdot \lambda^n / n!$$

return $pc(key)$

3. class: policy iteration

3.2 init (is solve 4-5 $\leftarrow 0$)

$$M_p \leftarrow 8$$

$$\delta_m \leftarrow 10^{-1}$$

$$\gamma \leftarrow 0.9$$

$$A \leftarrow \{-M_2, \dots, 0\} \cup \{0, \dots, M_1\}$$

$$\text{Map } V: S \rightarrow \mathbb{R}, V(S) \leftarrow 0$$

$$\text{Map } \pi: S \rightarrow A \subset \mathbb{Z}$$

is solve 4-5 $\leftarrow$ is solve 4-5

3.3 solve:

iterations $k \leftarrow 0$

loop:

$$V \leftarrow \text{policy evaluation}(V, \pi)$$

is policy change, $\pi \leftarrow$

policy improvement(A, V, π)

if is policy change = 0

stop

$$k \leftarrow k+1$$

3.4 (out-place)

policy evaluation(V, π)

process:

$$V_{\text{new}}: S \rightarrow \mathbb{R}$$

loop:

$$V_{\text{new}}(s) \leftarrow \gamma \sum_{a \in A} p_a(s) (R_a(s) + V(s)), \forall s \in S$$

$$\text{if } \|V_{\text{new}} - V\|_1 < \delta_m$$

$V \leftarrow V_{\text{new}}$, return V .

$$V \leftarrow V_{\text{new}}$$

3.5. policy improvement(A, V, π)

$$\pi_{\text{new}}: S \rightarrow A$$

expected action returns $qsa: S \times A \rightarrow \mathbb{R}$

$q_a(s, a) \leftarrow v \leftarrow q_{\pi, s, a} - p_i(V, s, a)$
 $\forall a \in A, s \in S.$

$\pi_{\text{new}}(s) \leftarrow \arg\max_a(q_a(s, :)), \forall s \in S.$

if $\pi_{\text{new}} \neq \pi^a$

$\pi \leftarrow \pi_{\text{new}}, \text{return } 1$

return 0 // is stable, is changed = 0.

3.6. // $O(n^4)$ for all req, ret.

Bellman(V, s, a) for $q(s, a)$

process:

$E_a \leftarrow 0$

if is solve 4-5

if $a > 0$

// free shuttle to loc 2.

$E_a \leftarrow E_a - (a-1)c_m$

else

$E_a \leftarrow E_a - |a|c_m$

else

$E_a \leftarrow E_a - |a|c_m$

$n_1 \leftarrow (s_1 - a) \wedge M_1$

$n_2 \leftarrow (s_2 + a) \wedge M_2$

$\forall \text{req}_1, \text{req}_2 \in \{0, \dots, M_p\}$

$p \leftarrow \text{poisson}(\text{req}_1, n_1) \cdot \text{poisson}(\text{req}_2, n_2)$

$n'_1 \leftarrow n_1, n'_2 \leftarrow n_2$

$n_{10} \leftarrow n'_1 \wedge \text{req}_1, n_{20} \leftarrow n'_2 \wedge \text{req}_2$

$r \leftarrow (n_{10} + n_{20})R_r$

if is solve 4-5

if $n'_1 \geq 10$

$r \leftarrow r - c_{\text{park}}$

if $n'_2 \geq 10$

$r \leftarrow r - c_{\text{park}}$

$n'_1 \leftarrow n'_1 - n_{10}, n'_2 \leftarrow n'_2 - n_{20}$

$E'_a \leftarrow 0$

$\forall \text{ret}_1, \text{ret}_2 \in \{0, \dots, M_p\}$

$p' \leftarrow \text{poisson}(\text{ret}_1, \lambda_{r1}) \cdot \text{poisson}(\text{ret}_2, \lambda_{r2})$

$n'_1 \leftarrow (n'_1 + \text{ret}_1) \wedge M_1$

$n'_2 \leftarrow (n'_2 + \text{ret}_2) \wedge M_2$

// classical Bellman equation

// for $V(s)$

$E'_a \leftarrow E'_a + p'V(s'),$

here $s' = (n'_1, n'_2)$

end $\forall \text{ret}_1, \text{ret}_2.$

$\therefore E_a \leftarrow E_a + p(r + \gamma E'_a)$

end $\forall \text{req}_1, \text{req}_2.$

return $E_a.$

3.7 $U_{\pi-s\pi}(V, s) // \pi(s)$

$a \leftarrow \pi(s)$

$E_a \leftarrow \text{Bellman}(V, s, a)$

return E_a .

3.8 $q_{\pi-s\pi}(V, s, a) // q_{\pi}(s, a)$

if $a \in [-s_2, 0] \cup [0, s_1]$

$E_a \leftarrow \text{Bellman}(V, s, a)$

else

$E_a \leftarrow -\infty$

return E_a .

4. test:

solver $\leftarrow$ Policy iteration (is solve $4.5 \leq 1$)

solver.solve

plot π .

grid world

1. $X_m \leq 4, Y_m \leq 4$

$A = \{(0, -1), (-1, 0), (0, 1), (1, 0)\}$

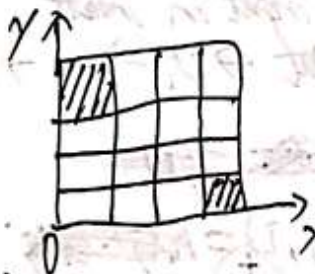
2. is terminal (s)

if $s = (1, Y_m)$ or $s = (X_m, 1)$

return 1

else

return 0.



3. step(s, a)

if is terminal (s)

return s, 0.

$s' \leftarrow s + a$

if $s'_1 < 1$ or $s'_1 > X_m$ or $s'_2 < 1$ or $s'_2 > Y_m$

$s' \leftarrow s$

$r \leftarrow -1$

return s', r

4. compute V (in place $\leftarrow 1, \gamma \leftarrow 1$)

Map $V: S \rightarrow \mathbb{R}$

$k \leftarrow 0$

if in place

$V_{old} \leftarrow V$

denote V as V_{new}

else

$V_{new} \leftarrow V$

denote V as V_{old}

loop:

if in place

$V_{old} \leftarrow V$

$\forall s \in S,$

$v \leftarrow 0$

$\forall a \in A,$

$s', r \leftarrow \text{step}(s, a)$

$v \leftarrow v + \underbrace{\gamma}_{0.75} (r + \gamma \cdot V(s'))$

end $\forall a$.

$V_{new}(s) \leftarrow v$

end $\forall s$.

$\Delta \leftarrow \max(|V_{old} - V_{new}|)$

if not in place

$V \leftarrow V_{new}$

if $\Delta < 10^{-4}$; stop loop

$k \leftarrow k + 1$, end loop.

return V, k.

π_1

Blackjack

1. $A = \{ \text{hit, stick} \}$
 $a_h \leftarrow 1, a_s \leftarrow 2$

5. $\pi_d: N \rightarrow A$
 $\text{Vec } N(21) \pi_d: \{1, \dots, 11\} \rightarrow a_h$
 $\{12, \dots, 16\} \rightarrow a_h$
 $\{17, \dots, 21\} \rightarrow a_s$

2. $\pi_p: N \rightarrow A$
 $\text{Vec } N(21) \pi_p: \{1, \dots, 11\} \rightarrow a_h$
 $\{12, \dots, 19\} \rightarrow a_h$
 $\{20, 21\} \rightarrow a_s$

6. $\text{get_card}()$
 $\text{card} \sim \text{uniform}(1, 13)$
 $\text{card} = \text{card} \wedge 10$
 return card

3. $\pi_p: S \rightarrow A$
 $\text{ask_ace_player } u_{ap}$
 $\text{player sum } s_p$
 $\text{dealer card } c_d$
 $\leftarrow s$

7. $\text{card_value}(\text{card})$
 $\text{if card} = 1 \text{ return } 11$
 else return card

8. $\text{play}(\pi, \text{initial } s, \text{initial } a)$
 $s_0 \leftarrow \text{none} \quad a_0 \leftarrow \text{none}$

4. $\mu_p: S \rightarrow A$
 $u_{ap}, s_p, c_d \leftarrow s$
 $\text{if binomial}(1, 0.5) = 1$
 $\text{return } a_s$
 $\text{else return } a_h$

// π is policy for player.
 $\text{sum of player } \sum p = 0$
 $\text{trajectory of player } \text{traj}_p \leftarrow \text{none}$
 $u_{ap} \leftarrow 0 \text{ (or 1)}$
 $\{ c_{id} = 0, c_{sd} = 0 \}$ (dealer status)
 $u_{ad} = 0$

f.3. $\text{binomial}(n, p)$
 $= \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}$

$\text{if } s_0 \text{ is none:}$
 $\text{while } \sum p < 12:$

$card \leftarrow \text{get card}$
 $sum_p \leftarrow sum_p + card_value(card)$
 if $card = 1$ if $sum_p > 21$
 $sum_p = sum_p - 10$
 $sum_p = sum_p - 10$ (if $card = 1$)
 $u_p = 1$ else
 if $card = 1$ and $u_p = 1$
 $u_p = 1$
 $u_p \leftarrow u_p + 1$ (if $card = 1$)

end while $s_p < 12$

$C_d \leftarrow \text{get card}$

$C_{2d} \leftarrow \text{get card}$

else // s_0 is not none.

$u_p, s_p, C_d \leftarrow s_0$

$C_{2d} \leftarrow \text{get card}$

assert $sum_p \leq 21$

~~$s_0 \leftarrow (u_p, sum_p, s_p, C_d)$~~

$sum_d \leftarrow card_value(C_d) + card_value(C_{2d})$

$is_1 = \{ is\ 1\ in\ \{C_d, C_{2d}\} \}$

if $sum_d > 21$

assert $sum_d = 22$ (L, $\because$ dealer only has 2 cards)

$sum_d = sum_d - 10$

~~assert $sum_p \leq 21$~~

// now we have $sum_d \in \{1, \dots, 21\}$

// game starts!

// player's turn.

loop: $s \leftarrow (u_p, sum_p, C_d)$

if initial action $a_0 \neq \text{none}$

~~$s \leftarrow (u_p, sum_p, C_d)$~~
 ~~$a \leftarrow a_0$~~

initial action ~~a_0~~ $\leftarrow \text{none}$

else

~~$s \leftarrow (u_p, sum_p, C_d)$~~
 ~~$a \leftarrow \pi_{\theta}(u_p, sum_p, C_d)$~~

$traj_p.add(s, a)$

if $a = a_s$: break loop

$card \leftarrow \text{get card}$

~~$count_ace \leftarrow \text{int}(u_p)$~~

if $card = 1$

~~$count_ace \leftarrow count_ace + 1$~~

~~$sum_p \leftarrow sum_p + card_value(card)$~~

$sum_p \leftarrow sum_p + \text{card value}(\text{card})$

if card = 1

if $uap = 1$

$sum_p = sum_p - 10$ (we can't have
~~two usable aces~~
more than one

else

$uap = 1$

if $sum_p > 21$

if $uap = 1$

$sum_p = sum_p - 10$

$uap = 0$

if $sum_p > 21$

return (s, 1, trajp)

end loop (player)

// dealer's turn.

loop:

$a \leftarrow \pi_d(sum_d)$

if $a = a_s$: break loop

card $\leftarrow$ get card(), $sum_d \leftarrow sum_d + \text{card value}(\text{card})$

if card = 1

if $uad = 1$: $sum_d \leftarrow sum_d - 10$

else: $uad = 1$

if $sum_d > 21$

if $uad = 1$

$sum_d \leftarrow sum_d - 10$

$uad = 0$

if $sum_d > 21$

return (s, 1, trajp)

end loop (dealer)

// compare sum_p and sum_d

(here we have $sum_p \leq 21$, $sum_d \leq 21$)

if $sum_p > sum_d$

return s, 1, trajp

else if $sum_p = sum_d$

return s, 0, 1

else

return s, 1, trajp

~~on-policy~~
~~with on-policy~~
~~random policy~~

breaking ties

random (see book)

9. MC sample with ~~on-policy~~
MC policy evaluation ~~with~~ (episodes)

$$\pi(s) \leftarrow \arg\max_a Q(s,a)$$

~~$V(s) \leftarrow 0, n(s) \leftarrow 1$~~ , $V(s)$
 $\forall i \in \{1, \dots, \text{episodes}\}$

end $V(s,a)$.
end $\forall i$.
return Q, π

~~G~~ , $\text{traj}_p \leftarrow \text{play}(\pi_p, \text{none}, \text{none})$

$\forall (s,a) \in \text{traj}_p$ // MC ~~off-policy~~ (episodes)

~~$n(s) \leftarrow n(s) + 1$~~

$$n(s) \leftarrow n(s) + 1$$

$$V(s) \leftarrow V(s) + \frac{1}{n(s)} (G - V(s))$$

MC ~~off-policy~~ estimate state value
- off-policy (episodes)

$$// V_{n+1}(s) = V_n(s) + \frac{n_h}{C_n} (G_n - V_n(s))$$

return V

10. MC with exploring starts

$$// V_{n+1}(s) = V_n(s) + \frac{1}{n} (W_n G_n - V_n(s))$$

MC-es (episodes)

$$s_0 = (\mu_p = 1, \text{sum}_p = 13, C_d = 2)$$

$$Q(s,a) \leftarrow 0$$

~~$n(s) \leftarrow 1$~~ , $V(s) \leftarrow 0$, $V_w \leftarrow 0$, $C \leftarrow 1$, OS, WS.

$$\pi \leftarrow \pi_p$$

~~$\forall i \in \{1, \dots, \text{episodes}\}$~~

$$n(s,a) \leftarrow 1$$

$G, \text{traj}_p \leftarrow \text{play}(\mu_p, s_0, \text{none})$

$\forall i \in \{1, \dots, \text{episodes}\}$

$\forall (s,a) \in \text{traj}_p$

$s_0 \leftarrow \text{random choice}(S)$

if $a = \pi_p(s)$

$a_0 \leftarrow \text{random choice}(A)$

$$W \leftarrow W \cdot \frac{1}{\pi_p(a|s)}, \text{ here } = 0.5$$

$G, \text{traj}_p \leftarrow \text{play}(\pi, s_0, a_0)$

else

$\forall (s,a) \in \text{traj}_p$

$W \leftarrow 0$, break $V(s,a)$

$$n(s,a) \leftarrow n(s,a) + 1$$

$$n \leftarrow n+1, C \leftarrow C+W$$

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{n(s,a)} (G - Q(s,a))$$

$$V_0 \leftarrow V_0 + \frac{1}{n} (W G - V_0)$$

$$V_w \leftarrow V_w + \frac{W}{C} (G - V_w)$$

return OS, WS

12. figure 5-1(c)

$V_1 \leftarrow$ MC policy evaluation on policy (10^4)

$V_2 \leftarrow$ MC policy evaluation on policy (5×10^5)

$V_1(x, :, u_{ap}=1). \log()$ 14. figure 5-3(1)

$V_1(x, :, u_{ap}=0). \log()$ true value = -0.27726

$V_2(x, :, u_{ap}=1). \log()$ episodes = 10^4

$V_2(x, :, u_{ap}=0). \log()$ runs = 100

13. figure 5-2(1)

$e_0 \leftarrow 0$ (episodes), $e_w \leftarrow 0$ (episodes)

$Q \leftarrow$ MC-es (5×10^5) $\forall i \in \{1, \dots, \text{runs}\}$

$V_*(s) \leftarrow \max_a Q(s, a), \forall s$

OS, NS $\leftarrow$ MC estimate state value
off-policy (episodes)

$f(x_1, x_2, x_3)$
 $g(x_1) = \max_{x_3} f(x_1, x_1, x_3)$
 $g_1(x_1, 1)$
 $g_2(x_1, 2)$

$f(x_1, x_2, x_3)$
 $g_1 = \max_{x_3} f(x_1, 1, x_3)$
 $g_2 = \max_{x_3} f(x_1, 2, x_3)$

$e_0 \leftarrow e_0 + (OS - \text{true value})^2$
// vector

$e_w \leftarrow e_w + (NS - \text{true value})^2$

$V_*(x, :, u_{ap}=1). \log(\text{"optimal value"})$ end $\forall i$

$V_*(x, :, u_{ap}=0). \log(\text{"optimal value"})$
without usable Acc.

$\pi_*(x, :, u_{ap}=1). \log(\text{"optimal policy"})$

$\pi_*(x, :, u_{ap}=0). \log(\text{"optimal policy"})$
without usable Acc.

$e_0 \leftarrow e_0 / \text{runs}, e_w \leftarrow e_w / \text{runs}$

plot(e_0),

plot(e_w)

title("ordinary importance sampling")

title("weight importance sampling")

π is deterministic policy

$$v_{\pi}(s)$$

$$v_{\pi}, q_{\pi}, \pi, q_{*}, q_{*}(s, a)$$

3.2 the backup diagram for MC estimation of v_{π} .

1. First-visit MC policy evaluation

$$\pi, V, \text{repeat } \{ \text{generate an episode using } \pi \}$$

Repeat:

generate an episode using π

$\forall s$ in the episode

$G \leftarrow$ return following the first occurrence of s

$$n(s) \leftarrow n(s) + 1$$

$$V(s) \leftarrow \frac{\sum G}{n(s)}$$

$$V(s) \leftarrow V(s) + \frac{1}{n(s)}(G - V(s))$$

$$V(s) \leftarrow \frac{\sum G}{n(s)}$$

$$E_d \leftarrow \dots$$

should use incremental implementation

4. Without a model, state values alone are not sufficient to determine a policy.

2. Blackjack: 2.2 playing blackjack is naturally formulated as an episodic finite MDP

2.3 consider the policy that sticks if the player's sum is 20 or 21, and otherwise hits.

2.4 to find v for this policy by a MC approach.

$$5. \pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1}$$

$$\rightarrow \dots \xrightarrow{I} \pi_{*} \xrightarrow{E} q_{*}$$

2.5 there is no difference between first-visit and every-visit MC methods.

2.6 the terminal reward is also the return.

3. entire trajectory of transitions along a single particular single episode

6. ~~MC control~~:

MC ES (MC control)

initialize $Q(s, a) \leftarrow \text{arbitrary}$

$\pi(s) \leftarrow \text{arbitrary}$

~~$n(s, a) \leftarrow 0$~~ , $n(s, a) \leftarrow 0$

~~$\text{returns}(s, a) \leftarrow \text{empty list}$~~

$\forall s, a \in S, a \in A(s)$

repeat:

① choose $s_0 \in S, a_0 \in A(s_0)$, s.t.

all pairs have probability > 0

② generate an episode starting from s_0, a_0 , the initial $Q(s, a) = 0$ following π .

③ $\forall$ pairs s, a in the episode

$G \leftarrow$ return following the first

occurrence of s, a .

$n(s, a) \leftarrow n(s, a) + 1$

$Q(s, a) \leftarrow Q(s, a) + \frac{1}{n(s, a)} (G - Q(s, a))$

$\pi(s) \leftarrow \arg\max_a Q(s, a)$

$Q(s, a) \leftarrow \frac{\sum G(s, a)}{n(s, a)}$

④ $\pi(s) \leftarrow \arg\max_a Q(s, a)$, $\forall s$ in the episode

7. solving Blackjack

7.2 apply MC ES to blackjack. with the sole exception of the leftmost notch

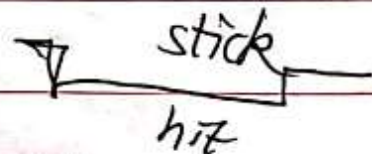
7.3: the episodes are all simulated games, ~~it is~~ it is easy to arrange for exploring starts that include all possibilities

7.4 pick s_0 at random with equal probability.

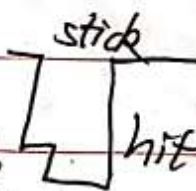
7.5 initial policy is the one in 2.

7.6 V is computed from Q found by one ES

usable ace:



No usable ace:



7.8 The policy is the same as the basic strategy of Thorp

in the policy for a usable acc,
which is not present in Thorp's
strategy. We are uncertain of
the reason for this discrepancy.

$$\begin{aligned} n(s,a) &\leftarrow n(s,a) + 1 \\ Q(s,a) &\leftarrow Q(s,a) + \frac{1}{n(s,a)} (G - Q(s,a)) \\ \text{sum } G &\leftarrow \text{sum } G + G \\ n(s,a) &\leftarrow n(s,a) + 1 \\ Q(s,a) &\leftarrow Q(s,a) + \frac{1}{n(s,a)} (G - Q(s,a)) \end{aligned}$$

~~incremental implementation~~

8. among ϵ -soft policies, ϵ -greedy policies are in some sense those that are closest to greedy.

$$\textcircled{3} \forall s \text{ in the episode } A^* \leftarrow \arg \max_a Q(s,a)$$

8.3 next page
9. I does not require that the policy be taken all the way to a greedy policy, only that

$$\pi(a|s) \leftarrow \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|} & \text{if } a = A^* \\ \frac{\epsilon}{|A(s)|} & \text{if } a \neq A^* \end{cases}$$

it be moved toward a greedy policy.

10. on-policy first-visit MC control (for ϵ -soft policies)

$\textcircled{1} (s,a) \leftarrow \text{arbitrary}$

~~Return (s,a) to empty list~~ $n(s,a) \leftarrow 0$

$\pi(a|s) \leftarrow$ an arbitrary ~~soft~~ ϵ -soft policy

Repeat:

$\textcircled{1}$ generate an episode using π

$\textcircled{2}$ for pair s,a in the episode

- $G \leftarrow$ return following the first ~~occurrence~~ of s,a

8.3 ~~ε~~ ϵ -soft policies π
 $\Leftrightarrow \pi(a|s) \geq \frac{\epsilon}{|A(s)|}, \forall s, a.$
 for some $\epsilon > 0$

8.4 ~~any~~ ϵ -soft policy π ,
 any ϵ -greedy policy with respect
 to q_π is guaranteed to be
 better than or equal to π .

3. ~~on-policy first-visit MC~~
 control (for ϵ -soft policies)
 init: $\pi(a|s)$, an arbitrary ϵ -soft policy.

10.4 ~~MC~~
 In the on-policy method we
 move it only to an ϵ -greedy policy.

~~off-policy~~ prediction via
 importance sampling

11.3 In practice, the weighted importance-sampling
 estimator usually has
 dramatically lower variance
 and is strongly preferred.

Repeat:

$$\pi(a|s) \in \left\{ 1 - \epsilon + \frac{\epsilon}{|A(s)|}, \frac{\epsilon}{|A(s)|} \right\}$$

10.3 here $\pi(a|s)$ is ϵ -soft policy,
 not ϵ -greedy policy.

~~10.5~~ if π' is partial ϵ -greedy policy,
 ~~$q_\pi(s, \pi'(s)) \geq u_\pi(s)$ if not ϵ -greedy~~
 ~~$q_\pi(s, \pi'(s)) \geq u_\pi(s)$ if ϵ -greedy~~

$$\Rightarrow q_\pi(s, \pi'(s)) \begin{cases} = u_\pi(s) & \text{if } \pi'(a|s) = \pi(a|s) \\ \geq u_\pi(s) & \text{if } \pi'(a|s) \text{ is } \epsilon\text{-greedy} \\ & \pi'(a|s) \neq \pi(a|s) \end{cases}$$

11.4 ~~MC~~

$$V_\pi(s) = \frac{\sum_{t \in J(s)} \gamma^t C_t}{|J(s)|}$$

 ordinary importance sampling.

11.5 weighted importance sampling:

$$V_\pi(s) = \frac{\sum_{t \in J(s)} \gamma^t C_t}{\sum_{t \in J(s)} \gamma^t}, \quad V_\pi(s) = 0 \text{ if } \sum_{t \in J(s)} \gamma^t = 0$$

11.6 $p_t^T = \dots = \frac{1}{T-t} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$

11.7 ~~12.~~ off-policy estimation of a
 blackjack state value.

12.2.

$$Q_\pi(s,a) = \frac{\sum_{t \in J(s,a)} \gamma^t C_t}{\sum_{t \in J(s,a)} \gamma^t}$$

12.2. We evaluate the state in which the dealer is showing a deuce, the sum of the player's cards is 13, and the player has a usable ace.

12.3 the data ~~was~~ is generated by starting in this state then choosing to hit or stick at random with equal probability (the behavior policy). The target policy is to stick only on a sum of 20 or 21.

$$V_3 = \frac{G_1 + G_2}{2}$$

$$\therefore V_{n+1} = \frac{G_{n+1} + G_n}{n}$$

$$\therefore n V_{n+1} = G_{n+1} + G_n$$

$$\therefore V_{n+1} = \frac{(n-1)V_n + G_n}{n}$$

$$= \frac{nV_n + G_n - V_n}{n}$$

$$= V_n + \frac{1}{n}(G_n - V_n)$$

13.3

$$\therefore V_{n+1} = \frac{\sum W_k G_k}{\sum W_k}, n \geq 1 \quad (C)$$

$$\therefore C_n V_{n+1} = \sum W_k G_k$$

$$= C_{n-1} V_n + W_n G_n, G_0 = 0$$

$$\therefore V_{n+1} = \frac{C_n V_n + W_n G_n - W_n V_n}{C_n}$$

$$\therefore = V_n + \frac{W_n}{C_n}(G_n - V_n)$$

(n ≥ 1)

here V_1 is arbitrary.

13 $V_{n+1} = V_n + \frac{1}{n}(G_n - V_n)$

$$n V_{n+1} = n V_n + G_n - V_n$$

$$\therefore V_{n+1} = \frac{G_n + (n-1)V_n}{n}$$

$$V_{n+1} = \frac{\sum_{k=1}^n G_k + G}{n}$$

B. $V_2 = \frac{W_1 a_1}{W_1} = G_1$

$$V_3 = \frac{W_1 a_1 + W_2 a_2}{W_1 + W_2}$$

13.4. let $W_n = 1, \therefore C_n = n$

$$\therefore V_{n+1} = V_n + \frac{1}{n}(G_n - V_n), n \geq 1$$

13.5 $V_{n+1} = \frac{\sum W_k G_k}{n}$

$$\Rightarrow V_{n+1} = V_n + \frac{1}{n}(W_n G_n - V_n)$$

$Q \rightarrow Q_n$
 If incremental off-policy (L) $\therefore$ If nongreedy actions are common, then learning will be slow, particularly for states appearing in the early portions of long episodes.
 every-visit MC policy evaluation.
 process: $Q(s,a) \leftarrow \text{arbitrary}, C(s,a) \leftarrow 0$
 $\mu(a|s) \leftarrow \text{an arbitrary soft behavior policy}$
 $\pi(a|s) \leftarrow \text{an arbitrary target policy}$
 $\forall s \in S, a \in A(s)$

Repeat:

1) generate an episode using μ :
 $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T$
 1.5 off-policy every-visit MC control (returns $\pi \approx \pi_*$)

2) $Q \leftarrow 0$
 3) $W \leftarrow 1$
 4) $\forall t = T-1, T-2, \dots, 0$
 $Q \leftarrow \gamma Q + R_{t+1}$
 $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} (G - Q(S_t, A_t))$
 $W \leftarrow W \cdot \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}$
 process:
 $Q(s,a) \leftarrow \text{arbitrary}, C(s,a) \leftarrow 0$
 $\pi(s) \leftarrow \text{a deterministic policy that is greedy w.r.t. } Q$

If $W=0$: exit for loop.

loop:
 ① generate an episode using any soft policy μ :
 $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T$

1.5 off-policy MC control

1.5.3 π converges to optimal at all encountered states.

1.5.4 $\therefore$ off-policy MC control method

learns only from the tails of episodes.

② $Q \leftarrow 0$

③ $W \leftarrow 1$

④ $\forall t = T-1, T-2, \dots, 0$

$Q \leftarrow \gamma Q + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} (G - Q(S_t, A_t))$$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken consistently)

if $A_t \neq \pi(S_t)$ then exit for loop.

$$W \leftarrow W \frac{1}{\mu(A_t|S_t)}$$

~~speed up convergence~~

~~speed up convergence~~

(ϵ)
why put
here, not
last

random walk

true value v_π .

1. $v_\pi(C) = 0.5$

$v_\pi(i) = \frac{i}{6}, i = 1, \dots, 5$



0 1 2 3 4 5 6 $s \in S$

1.3 $V(s) = 0.5, V(0) = 0, V(6) = 0$

1.4 $S = \{1, \dots, 5\}$, all episodes start in S

2. $v_\pi: S \rightarrow R$, (1.5) start in S

true value

$v_\pi(s) = \frac{s}{6}, \bar{s} = 1, \dots, 5, v_\pi(0) = 0, v_\pi(6) = 0$

2.3 action left $a_l = 0$

action right $a_r = 1$

3. temporal difference ~~TD(0)~~ TD(0)

($V, \alpha = 0.1, is_batch = 0$)

state $s = 3$.

trajectory traj add s .

~~rewards add r~~

loop:

$x \sim \text{binomial}(1, 0.5)$

if $x = a_l$

$s' = s - 1$

else $s' = s + 1$

~~reward r~~ $\leftarrow$ if $s' = 6, r = 1$ else $r = 0$ ~~assume all rewards are 0~~

1.5 All episodes start in C , and proceed either left or right by one state on each step, with equal probability.

~~traj add s~~

if $is_batch = 0$

$V(s) \leftarrow V(s) + \alpha \cdot (r + V(s') - V(s))$

$s \leftarrow s'$ if $s \neq s'$ traj add (s) , $rs.add(r)$

if $s = 6$ or $s = 0$

break loop

~~rewards add r~~

end loop

return traj, rewards.

4. constant- α MC

($V, \alpha = 0.1, is_batch \neq 0$)

$s = 3$

traj add s .

loop:

$x \sim \text{binomial}(1, 0.5)$

if $x = a_1$

$s' = s - 1$

else

$s' = s + 1$

~~try add s'~~

~~if $s' \neq 0$~~

~~return $G = 1$~~

~~else if $s = 0$~~

~~$G = 0$~~

~~break loop~~

~~$s = s'$~~

~~end loop~~

if is batch = 0

$V(s) \leftarrow V(s) + \alpha \cdot (G - V(s))$, $\forall s \in \text{traj}$, $\text{method} \leftarrow \text{MC}$

return traj, ~~G~~ , $\text{rs.setSize}(\text{traj})$, $\text{rs} \leftarrow G$.

5. example 6.2 left.

compute state value:

episodes = {0, 1, 10, 100}

$V \leftarrow V_0$

$\forall i \in \{0, \dots, 100\}$

if $i \in \text{episodes}$

plot V , with title: $i + \text{'episodes'}$.

$\text{TD}(0)(V)$
end $\forall i$

plot v_π .

6 example 6.2 right

$\text{rms_error}()$:

~~$\text{TD}(0)$~~ $\alpha s = \{0.15, 0.1, 0.05\}$

~~MC~~ $\alpha s = \{0.01, 0.02, 0.03, 0.04\}$

episodes = ~~100~~ / 0

runs = 100, total errors $E = 0(\text{episode})$

$\forall i \in \{1, \dots, m\}$, $\alpha s = \{\text{TD}(0) \alpha s, \text{MC} \alpha s\}$

$\alpha \leftarrow \alpha s(i)$

if $i \leq |\text{TD}(0) \alpha s|$

method $\leftarrow \text{TD}$

else

method $\leftarrow \text{MC}$

$r \leftarrow |runs| = \{1, \dots, m, |runs|\}$

errors $\leftarrow R$

$V \leftarrow V_0$

$\forall j \in \{1, \dots, m, \text{episodes}\}$

$\text{err} \leftarrow \sqrt{\frac{\sum (V_j - v_\pi)^2}{\text{size}}}$, here = 5

errors.add(err)

if method = "TD"

$\text{TD}(0)(V, \alpha)$

else

$\text{MC}(V, \alpha)$

end $\forall i$

total_errors $\leftarrow$ total_errors + errors.
end $\forall r$

total_errors $\leftarrow$ total_errors / runs

plot(total_errors, name: method + α)

7. // figure 6.3

batch_updating(method, episodes, $\alpha=10^{-3}$)

runs = 100

total_errors $E \leftarrow 0$ (episodes)

$\forall r \in \{1, \dots, \text{runs}\}$

$V \leftarrow V_0$

errors $\leftarrow R^{\text{episodes}}$

trajs, rewards $rs \leftarrow \emptyset$

$\forall ep \in \{1, \dots, \text{episodes}\}$

if method = 'TD'

traj, rs $\leftarrow$ TD0(V , is_batch=1)

else

traj, rs $\leftarrow$ constant- α MC(V , is_batch=1)

trajs $\leftarrow$ add(traj)

rs $\leftarrow$ add(rs)

loop

$\sum \delta \leftarrow 0$ (size)

$\forall (traj, rs) \text{ in } (trajs, rs)$

$\forall i \in \{1, \dots, |traj| - 1\}$

if method = 'TD'

$\delta \leftarrow V^i + (rs(i) + V(traj(i+1)) - V(traj(i)))$
See 10
else

$V^i \leftarrow V^i + (rs(i) - V(traj(i)))$

end $\forall i$, end $\forall (traj, rs)$

if $\|\sum \delta\| < 1$

break loop.

$V \leftarrow V + \alpha \cdot \sum \delta$

end loop.

err $\leftarrow \sqrt{\frac{\sum (V^i - V)^2}{S.size()}}$, here 5.

errors.add(err)

end $\forall ep$.

$E \leftarrow E + errors$

end $\forall r$.

$E \leftarrow \frac{E}{runs}$

return E.

8. example 6-2()

~~rms_error()~~

compute_state_value()

rms_error()

9. figure 6-3()

episodes = 100

TD_errors ← batch Updating ('TD', episodes)

MC_errors ← batch Updating ('MC', episodes)

plot(TD_errors)

plot(MC_errors)

11. if $s' = 6$

$A \leftarrow 1$

else if $s' = 0$

$A \leftarrow 0$

~~else $A \leftarrow 0$~~

$s \leftarrow s'$

traj.add(s), ~~return traj~~

if $s = 0$ or $s = 6$

break loop

end loop

~~if is batch~~

10. $s \leftarrow \text{traj}(i)$, $s' \leftarrow \text{traj}(i+1)$, ~~$r \leftarrow \text{traj}(i+1)$~~

~~$\delta \leftarrow r + V(s') - V(s)$~~
 $\text{sum_}\delta(s) \leftarrow \text{sum_}\delta(s) + \delta$

if method = "TD"

~~$r \leftarrow \text{traj}(i)$~~
 $\delta \leftarrow r + V(s') - V(s)$

else

~~$\delta \leftarrow r + V(s') - V(s)$~~

$A \leftarrow r + V(s)$

$\delta \leftarrow A - V(s)$

$\text{sum_}\delta(s) \leftarrow \text{sum_}\delta(s) + \delta$

TD(0)

1. $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$ // $Q(s,a): s \in S^+, a \in A(s)$
 $V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$ Repeat for each episode:

2. Tabular TD(0) for estimating v_π

Input: π

initialize $V(s)$ arbitrarily, $\forall s \in S^+$
 eg. $V(s) = 0, \forall s \in S^+$

Process:

loop for each episode

initialize $S \in S$

repeat for each step of episode

$A \sim \pi(A|S)$

take A , observe R, S'

$V(S) \leftarrow V(S) + \alpha(R + \gamma V(S') - V(S))$ $S \leftarrow S', A \leftarrow A'$

$S \leftarrow S'$

until S is terminal.

end loop.

① initialize $S \in S$

② choose A from S using policy derived from Q (e.g., ϵ -greedy)

③ Repeat (for each step of episode)

take A , observe R, S'

choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S,A) \leftarrow Q(S,A) + \alpha(R + \gamma Q(S',A) - Q(S,A))$

④ until S is terminal.

⑤ end repeat for each episode.

$$3 \text{ ① } G_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$2 \text{ ② } G_t - V(S_t) = \sum_{k=0}^{T-t-1} \gamma^k \delta_{t+k}, \text{ here}$$

$$G_T = 0, V(S_T) = 0.$$

$$6. Q(S_t, A_t) \leftarrow Q(S_t, A_t)$$

$$+ \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$$

$$4. Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

5. Sarsa: an on-policy TD control algorithm.

initialize $Q(s,a)$ arbitrarily, $\forall s \in S, a \in A(s)$. $Q(s, \cdot) = 0, \forall s$ is terminal state.

7. Q-learning: an off-policy TD control algorithm.

Initialize S

Repeat for each step of episode

initialize $Q(s, a)$ arbitrarily, $\forall s \in S, a \in A(s)$, $\textcircled{1}$ Choose A from S using

$$Q(s, \cdot) = 0, \forall s \in S' / S$$

policy derived from Q_1, Q_2 .

(e.g., ϵ -greedy in $Q_1 + Q_2$)

Repeat for each episode:

initialize $S \in S$

$\textcircled{2}$ Take A , observe R, S'

repeat for each step of episode:

$\textcircled{3}$ With 0.5 probability:

choose A from S using policy derived from Q , $Q_1(S, A) \leftarrow Q_1(S, A)$

(e.g., ϵ -greedy)

$$+ \alpha (R + \gamma Q_2(S', \arg \max_a Q_1(S', a)) - Q_1(S, A))$$

take A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma \max_a Q(S', a) - Q(S, A))$$

$$Q_2(S, A) \leftarrow Q_2(S, A)$$

$$+ \alpha (R + \gamma Q_1(S', \arg \max_a Q_2(S', a)) - Q_2(S, A))$$

$$S \leftarrow S'$$

until S is terminal.

$\textcircled{4} S \leftarrow S'$

end Repeat for each episode.

$$8. Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma E(Q(S_{t+1}, A_{t+1}) | S_{t+1}) - Q(S_t, A_t))$$

$$\leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma \sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t))$$

$$9. Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha (R_{t+1} + \gamma Q_2(S_{t+1}, \arg \max_a Q_1(S_{t+1}, a)) - Q_1(S_t, A_t))$$

10. double Q-learning

Initialize $Q_1(s, a), Q_2(s, a)$ arbitrarily, $\forall s \in S, a \in A(s)$.

$$Q_1(s, \cdot) = Q_2(s, \cdot) = 0, \forall s \in S' / S$$

$\textcircled{5}$ until S is terminal

Repeat for each episode:

end repeat for each episode.

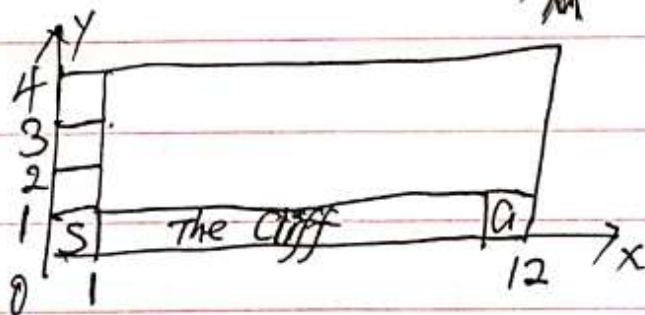
Cliff walking

1. ~~$X_M = 12$~~ , $X_M = 12$, $\epsilon = 0.1$, $\text{ALPHA} = 0.5$, $\gamma = 1$.

$a_l, a_r, a_u, a_d \in \{1, 2, 3, 4\}$

$A = \{a_l, a_r, a_u, a_d\}$

START $s = (1, 1)$, GOAL $= (12, 1)$



2. step (state s , action a):

assert $s \neq \text{GOAL}$.

$x, y \leftarrow s$

if $a = a_l$: $x \leftarrow x - 1$

else if $a = a_r$: $x \leftarrow x + 1$

else if $a = a_u$: $y \leftarrow y + 1$

else $y \leftarrow y - 1$

if $x < 1$: $x \leftarrow 1$

else if $x > X_M$: $x \leftarrow X_M$

if $y < 1$: $y \leftarrow 1$

else if $y > Y_M$: $y \leftarrow Y_M$

$s' \leftarrow (x, y)$

$r \leftarrow -1$

~~if $s \neq \text{GOAL}$ and $1 \leq x \leq X_M$ and~~

if $(x, y) \in [1, X_M] \times [1, Y_M]$

$s' \leftarrow \text{START}$, $r \leftarrow -100$

return r, s' .

~~$Q(s, a) \in \mathbb{R}$~~

~~$Q(s, a) \leftarrow -1, \forall s, a$~~

~~$Q(s, a) \leftarrow -100, s \in [1, X_M] \times [1, Y_M]$~~

~~$Q(s, a) \leftarrow -100, s = \text{START}, a = a_r$~~

4 choose action (s, Q)

~~$\text{sample} \leftarrow \text{binomial}(1, \epsilon)$~~

if sample = 1

~~draw~~

$i_a \sim \text{uniform}(1, 4)$

$a \leftarrow A(i_a)$

else

~~$Q(s, a) \leftarrow Q(s, a)$~~

$a \leftarrow \underset{a}{\text{argmax}} Q(s, a)$ randomly

return a .

5. ~~$\text{sarsa}(Q, \text{is_expected} \leftarrow 0,$~~

$\alpha \leftarrow \text{ALPHA}$

$s \leftarrow \text{START}$

$a \leftarrow \text{choose action}(s, Q)$

sum $r \leftarrow 0$

loop:

$r, s' \leftarrow \text{step}(s, a)$

$a' \leftarrow \text{choose_action}(s', Q)$

$\text{sum_r} \leftarrow \text{sum_r} + r$

if $r \text{ is expected} = 0$

~~Q~~ $\leftarrow Q(s', a')$

else

~~Q~~ $\leftarrow 0$

$\text{set_a} \leftarrow \text{argwhere}(Q(s', :) = \max_a Q(s', :))$

$\forall a \in \text{set_a}$

if $a \in \text{set_a}$

$p \leftarrow \frac{1-\epsilon}{|\text{set_a}|} + \frac{\epsilon}{|A|}$

~~Q~~ $\leftarrow \text{del} + p \cdot Q(s', a)$

else

$p \leftarrow \frac{\epsilon}{|A|}$

$\text{del} \leftarrow \text{del} + p \cdot Q(s', a)$

end if

$\text{del} \leftarrow \text{del} + r + \gamma Q$

$Q(s, a) \leftarrow Q(s, a) + \alpha (Q - Q(s, a))$

$s \leftarrow s', a \leftarrow a'$

if $s = \text{GOAL}$: stop loop.

end loop.

return sum_r .

6. Q-learning($Q, \alpha = \text{ALPHA}$)

$s \leftarrow \text{START}$

$\text{sum_r} \leftarrow 0$

loop:

$a \leftarrow \text{choose_action}(s, Q)$

$r, s' \leftarrow \text{step}(s, a)$

$\text{sum_r} \leftarrow \text{sum_r} + r$

$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$

$s \leftarrow s'$

if $s = \text{GOAL}$

stop loop

end loop

return sum_r .

7. print_optimal_policy(Q)

7. print_optimal_policy(Q) $\text{sum_rs_sarsa} \neq \text{runs}$

$\pi(s) = \underset{a}{\operatorname{argmax}} Q(s, a), \forall s.$

$\text{sum_rs_Q_learning} \neq \text{runs}.$

$\text{plot}(\text{sum_rs_sarsa}, \text{name} = "sarsa")$

$\text{plot}(\text{sum_rs_Q_learning}, \text{name} = "Q_learning")$

print π

i.e. ~~if $\pi(s) \neq a$~~ $a \leftarrow \pi(s)$

if $s = \text{GOAL}$, print "G", continue.

if $a = a_u$: print: " $\leftarrow$ "

else if $a = a_r$: print: " $\rightarrow$ "

else if $a = a_u$: print: " $\uparrow$ "

else: print " $\downarrow$ "

save figure("sum_r")

print_optimal_policy(Q_sarsa)

print_optimal_policy(Q_Q_learning)

8. figure 6.4()

episodes = 500

runs = 50

$\text{sum_rs_sarsa} \leftarrow \text{episodes}$

$\text{sum_rs_Q_learning} \leftarrow \text{episodes}$

$\forall \text{run} \in \{1, \dots, \text{runs}\}$

$Q_sarsa \leftarrow R^{X_m \times Y_m \times 4}$

$Q_Q_learning \leftarrow R^{X_m \times Y_m \times 4}$

$Q_sarsa \leftarrow 0, Q_Q_learning \leftarrow 0$

$\forall i \in \{1, \dots, \text{episodes}\}$

$\text{sum_rs_sarsa}(i) \leftarrow \text{sum_rs_sarsa}(i) + \text{sarsa}(Q_sarsa)$

$\text{sum_rs_Q_learning}(i) \leftarrow \text{sum_rs_Q_learning} + Q_learning(Q_Q_learning)$

end $\forall i, \text{run}$

end $\forall \text{run}$

9. figure 6.6()

episodes = 1000, runs = 10

$\alpha_s \in \{0.1, 0.2, \dots, 1\}$

asymptotic_sarsa, $\alpha/sarsa \leftarrow 1$

$\alpha/\text{expected_sarsa} \leftarrow$

$\alpha/\text{Q_learning} \leftarrow$

interim ~~intermediate~~ ~~intermediate~~ sarsa $\leftarrow 4$

~~interim~~ ~~intermediate~~ ~~intermediate~~ expected_sarsa $\leftarrow$

interim ~~intermediate~~ Q_learning $\leftarrow 6$

performance $\leftarrow 0^{6 \times |os|}$

$\forall run \in \{1, m, runs\}$

$\forall i \in \{1, m, |os|\}$

$\alpha \leftarrow \alpha_s(i)$

$Q_sarsa \leftarrow 0^{X_{max} \times Y_{max} \times 4}$

$Q_expected_sarsa \leftarrow 0^{X_{max} \times Y_{max} \times 4}$

$Q_Q_learning \leftarrow 0^{X_{max} \times Y_{max} \times 4}$

$\forall ep \in \{1, m, episodes\}$

sum_rs_sarsa $\leftarrow$ sarsa($Q_sarsa, 1, \alpha$)

sum_rs_expected_sarsa $\leftarrow$ sarsa($Q_expected_sarsa, 1, \alpha$)

sum_rs_Q_learning $\leftarrow$ Q-learning($Q_Q_learning, \alpha$)

performance(asy-sarsa, i) $+=$ sum_rs_sarsa

performance(asy-expected-sarsa, i) $+=$ sum_rs_expected_sarsa

performance(asy-Q-learning, i) $+=$ sum_rs_Q_learning

if $ep \leq 100$

performance(~~if~~ ^{interim} sarsa, i) $+=$ sum_rs_sarsa

...

...

end $\forall ep, i, run$

performance(1:3, :) $/=$ episodes $\cdot$ runs

(4:6, :) $/=$ 100 $\cdot$ runs

plot(performance, name='sum_reward per episode')